QueueSmart

Assignment 4: Data Design & Database Implementation

QueueSmart - Smart Queue Management Application

Group 13

Raqib Ayodeji Fagbayi | Ahmad Kaseb | Nick Edwards Mayes | Ricardo Rios Hernandez

- SQLite relational database integrated with the Assignment 3 FastAPI backend
- Assignment 2 front-end screens now retrieve and display persistent data
- 33 unit/API/database tests passed with 92% backend code coverage
- Prepared for Canvas soft-copy submission

IMPORTANT: Replace the GitHub placeholder in Section 1 after pushing the final code.

1. GitHub Repository Link

Repository Link: https://github.com/YOUR-USERNAME/queuesmart

Required final action: replace the highlighted placeholder with the actual URL of the existing QueueSmart repository after the Assignment 4 code is pushed. The TAs will validate database integration, tests, and each member's work using GitHub commit history.

2. Database Choice and Alignment with Assignments 1-3

QueueSmart uses SQLite as a relational database through SQLAlchemy 2. This keeps the same Python/FastAPI backend from Assignment 3 and the same plain HTML, CSS, and JavaScript front end from Assignment 2. SQLite is appropriate for the course project because the application has structured relationships between users, services, queues, queue entries, notifications, and history, while still being simple for every team member and TA to run locally.

Technology	Reason Used	How It Supports QueueSmart
SQLite	Provides a real persistent relational database without requiring a separate database server.	Stores all required application data across requests and server restarts.
SQLAlchemy 2	Defines models, foreign keys, relationships, transactions, and parameterized queries.	Connects the existing FastAPI routes to SQLite and reduces SQL-injection risk.
FastAPI + Pydantic	Preserves the Assignment 3 API structure and validates incoming request data.	Enforces required fields, correct types, length limits, ranges, and roles before database writes.
PBKDF2-SHA256	Uses a salted, one-way password hash from the Python standard library.	Prevents plain-text password storage and satisfies the password-protection requirement.

3. Database Schema and Relationships

The database uses normalized relational tables. UserProfile references UserCredentials by user_id instead of duplicating email. Email remains the unique username in UserCredentials and is available through the one-to-one relationship. Notification and History are implemented as separate tables so QueueSmart can store both user messages and completed queue participation records.

Table	Primary Key	Important Foreign Keys / Relationships	Required Data and Constraints
user_credentials	id	Parent of profile, queue entries, notifications, history, and sessions.	Unique email; salted password_hash; role user/administrator; email length check.
user_profiles	id	user_id -> user_credentials.id (unique, cascade delete).	Full name required; optional contact information and preferences with length limits.
services	id	Parent of queues and history records.	Unique name; description; duration 1-120; priority low/medium/high.
queues	id	service_id -> services.id	Status open/closed; created

Table	Primary Key	Important Foreign Keys / Relationships	Required Data and Constraints
		(cascade delete).	date.
queue_entries	id	queue_id -> queues.id; user_id -> user_credentials.id.	Position >= 1; join time; reason length; status waiting/served/canceled.
notifications	id	user_id -> user_credentials.id (cascade delete).	Message, timestamp, status sent/viewed.
history	id	user_id, service_id, and queue_entry_id foreign keys.	Outcome served/canceled; completed time; nonnegative wait minutes.
session_tokens	token	user_id -> user_credentials.id (cascade delete).	Persistent authentication token and creation time.

3.1 SQL Statements

The complete executable schema is also included in backend/schema.sql and docs/database_schema.sql. Primary keys, foreign keys, unique constraints, check constraints, and indexes are shown below.

```
PRAGMA foreign_keys = ON;


CREATE TABLE user_credentials (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    email VARCHAR(254) NOT NULL UNIQUE,
    password_hash VARCHAR(512) NOT NULL,
    role VARCHAR(20) NOT NULL DEFAULT 'user',
    created_at DATETIME NOT NULL,
    CONSTRAINT ck_user_email_length CHECK (length(email) BETWEEN 3 AND 254),
    CONSTRAINT ck_user_role CHECK (role IN ('user', 'administrator'))
);
CREATE INDEX ix_user_credentials_email ON user_credentials(email);


CREATE TABLE user_profiles (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL UNIQUE,
    full_name VARCHAR(80) NOT NULL,
    contact_info VARCHAR(120),
    preferences VARCHAR(250),
    CONSTRAINT fk_profile_user FOREIGN KEY (user_id)
        REFERENCES user_credentials(id) ON DELETE CASCADE,
    CONSTRAINT ck_profile_name_length CHECK (length(full_name) BETWEEN 2 AND 80),
    CONSTRAINT ck_profile_contact_length CHECK (contact_info IS NULL OR length(contact_info) <= 120),
    CONSTRAINT ck_profile_preferences_length CHECK (preferences IS NULL OR length(preferences) <= 250)
);


CREATE TABLE services (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(100) NOT NULL UNIQUE,
    description VARCHAR(250) NOT NULL,
    expected_duration INTEGER NOT NULL,
    priority_level VARCHAR(10) NOT NULL,
    created_at DATETIME NOT NULL,
    CONSTRAINT ck_service_name_length CHECK (length(name) BETWEEN 2 AND 100),
    CONSTRAINT ck_service_description_length CHECK (length(description) BETWEEN 5 AND 250),
    CONSTRAINT ck_service_duration CHECK (expected_duration BETWEEN 1 AND 120),
    CONSTRAINT ck_service_priority CHECK (priority_level IN ('low', 'medium', 'high'))
);


CREATE TABLE queues (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    service_id INTEGER NOT NULL,
    status VARCHAR(10) NOT NULL DEFAULT 'open',
```

```
    created_at DATETIME NOT NULL,
    CONSTRAINT fk_queue_service FOREIGN KEY (service_id)
        REFERENCES services(id) ON DELETE CASCADE,
    CONSTRAINT ck_queue_status CHECK (status IN ('open', 'closed'))
);
CREATE INDEX ix_queues_service_id ON queues(service_id);


CREATE TABLE queue_entries (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    queue_id INTEGER NOT NULL,
    user_id INTEGER NOT NULL,
    position INTEGER NOT NULL,
    join_time DATETIME NOT NULL,
    completed_at DATETIME,
    status VARCHAR(10) NOT NULL DEFAULT 'waiting',
    reason_for_visit VARCHAR(200) NOT NULL,
    CONSTRAINT fk_entry_queue FOREIGN KEY (queue_id)
        REFERENCES queues(id) ON DELETE CASCADE,
    CONSTRAINT fk_entry_user FOREIGN KEY (user_id)
        REFERENCES user_credentials(id) ON DELETE CASCADE,
    CONSTRAINT ck_queue_entry_position CHECK (position >= 1),
    CONSTRAINT ck_queue_entry_status CHECK (status IN ('waiting', 'served', 'canceled')),
    CONSTRAINT ck_queue_entry_reason_length CHECK (length(reason_for_visit) BETWEEN 2 AND 200),
    CONSTRAINT uq_queue_entry_join UNIQUE (queue_id, user_id, join_time)
);
CREATE INDEX ix_queue_entries_queue_id ON queue_entries(queue_id);
CREATE INDEX ix_queue_entries_user_id ON queue_entries(user_id);


CREATE TABLE notifications (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    message VARCHAR(300) NOT NULL,
    timestamp DATETIME NOT NULL,
    status VARCHAR(10) NOT NULL DEFAULT 'sent',
    CONSTRAINT fk_notification_user FOREIGN KEY (user_id)
        REFERENCES user_credentials(id) ON DELETE CASCADE,
    CONSTRAINT ck_notification_message_length CHECK (length(message) BETWEEN 1 AND 300),
    CONSTRAINT ck_notification_status CHECK (status IN ('sent', 'viewed'))
);
CREATE INDEX ix_notifications_user_id ON notifications(user_id);


CREATE TABLE history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    service_id INTEGER NOT NULL,
    queue_entry_id INTEGER NOT NULL UNIQUE,
    joined_at DATETIME NOT NULL,
    completed_at DATETIME NOT NULL,
    outcome VARCHAR(10) NOT NULL,
    wait_minutes INTEGER NOT NULL DEFAULT 0,
    CONSTRAINT fk_history_user FOREIGN KEY (user_id)
        REFERENCES user_credentials(id) ON DELETE CASCADE,
    CONSTRAINT fk_history_service FOREIGN KEY (service_id)
        REFERENCES services(id) ON DELETE RESTRICT,
    CONSTRAINT fk_history_entry FOREIGN KEY (queue_entry_id)
        REFERENCES queue_entries(id) ON DELETE CASCADE,
    CONSTRAINT ck_history_outcome CHECK (outcome IN ('served', 'canceled')),
    CONSTRAINT ck_history_wait_minutes CHECK (wait_minutes >= 0)
);
CREATE INDEX ix_history_user_id ON history(user_id);
CREATE INDEX ix_history_service_id ON history(service_id);


CREATE TABLE session_tokens (
    token VARCHAR(128) PRIMARY KEY,
    user_id INTEGER NOT NULL,
    created_at DATETIME NOT NULL,
    CONSTRAINT fk_session_user FOREIGN KEY (user_id)
        REFERENCES user_credentials(id) ON DELETE CASCADE
);
CREATE INDEX ix_session_tokens_user_id ON session_tokens(user_id);
```

4. Backend and Database Validations

Validation is enforced in two layers: Pydantic validates API requests before business logic runs, and SQLite CHECK, UNIQUE, NOT NULL, and FOREIGN KEY constraints protect stored data. SQLAlchemy sends values as parameters instead of building SQL with string concatenation.

Requirement	Implementation
Required fields	NOT NULL database columns and Pydantic required request fields for users, services, and queue actions.
Field length limits	Full name 2-80; email 3-254; password request 8-72; service name 2-100; description 5-250; reason 2-200; optional profile fields limited.
Correct data types and ranges	Integer IDs; service duration 1-120; queue position >= 1; Literal validation for role, priority, queue status, entry status, and notification status.
Unique constraints	Unique email, service name, one profile per user, and one history record per queue entry.
Foreign keys	SQLite foreign-key enforcement is enabled using PRAGMA foreign_keys=ON for every connection.
Password protection	Salted PBKDF2-SHA256 hashes are stored in password_hash. The database never stores plain-text passwords.
Authorization	Bearer session tokens plus user-only and administrator-only dependencies protect role-specific APIs.

5. Data Persistence, Retrieval, and UI Display

The in-memory lists and dictionaries from Assignment 3 were replaced by SQLAlchemy queries and SQLite transactions. The database file is created automatically at backend/queuesmart.db when the server starts. Data is committed before a successful API response is returned, so later requests and new database sessions can retrieve the same records.

Application Feature	Database Write	Database Retrieval / UI Display
Registration and login	Creates UserCredentials and UserProfile; stores password hash and session token.	Login returns user role/name; header and navigation update from backend response.
Service management	Administrators insert and update Service rows and open/close Queue rows.	User/admin screens list available services, durations, priorities, status, and current waiting counts.
Join/leave queue	Creates or updates QueueEntry; positions are committed and renumbered transactionally.	Queue Status and User Dashboard show current position, service, status, and estimated wait.
Serve/remove/reorder	Administrator actions update QueueEntry, position order, Notification, and History records.	Queue Management displays the selected service queue and waiting users from the database.
Notifications	Join, almost-ready, served, leave, and removal events create Notification rows.	User Dashboard retrieves and displays notification messages and sent/viewed status.

Application Feature	Database Write	Database Retrieval / UI Display
History	Served or canceled entries create relational History records linked to user, service, and entry.	History screen displays date, service name, wait time, and outcome.

5.1 Preserved Assignment 3 API Structure

Module	Main Endpoints
Authentication/Profile	POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET/PUT /api/profile
Services	GET /api/services, POST /api/services, PUT /api/services/{id}, POST /api/services/{id}/queue/toggle
Queues / Wait Time	POST /api/queues/join, DELETE /api/queues/{queue_id}/leave, GET /api/queues/status, GET /api/services/{id}/estimate
Notifications / History	GET /api/notifications, POST /api/notifications/{id}/view, GET /api/history
Administrator Queue Tools	GET /api/admin/dashboard, GET /api/admin/queues/{service_id}, serve-next, remove, and move endpoints

6. Unit Tests and Code Coverage

All new database integration code is covered by unit/API tests. Tests use a fresh temporary SQLite database so they verify real table creation, commits, relationships, retrieval across separate requests, role authorization, and validation without changing the development database.

Tests Passed	Failures	Backend Coverage	Required Target	Storage Tested
33	0	92%	70-80%	SQLite relational database

- Authentication: registration, salted password hashing, login, invalid credentials, unique email, sessions, logout, profile retrieval/update, and validation.
- Services: database retrieval, create/update, unique name, role checks, duration/priority validation, wait estimates, and open/close queue persistence.
- Queues: join, duplicate prevention, closed queue handling, leave, serve next, remove, reorder, position renumbering, notifications, history, and dashboard counts.
- Persistence: committed records are visible from a new SQLAlchemy session and a separate HTTP request; relational history links are verified.

Coverage Report Evidence

QueueSmart - Test Coverage

```
$ coverage run -m pytest
33 passed

$ coverage report -m
Name                  Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
backend/database.py      43      3      6      2    90%   44->47, 64->exit, 73-76
backend/main.py         305     22     66     19    89%   58, 72, 112->118, 152->154, 155, 200-202, 229->232,
backend/models.py        92      0      0      0   100%
backend/schemas.py       31      0      0      0   100%
backend/security.py      24      3      2      1    85%   36, 41-42
backend/seed.py          16      0      6      0   100%
-----------------------------------------------------------------
TOTAL                   511     28     80     22    92%
```

Figure 1. coverage.py output showing 33 passing tests and 92% total backend coverage.

7. Team Contributions

All four members must push meaningful commits from their own GitHub accounts. The commit history should match the work below; a contribution listed only in the document is not enough for grading.

Group Member Name	What is your contribution?	Discussion Notes
Raqib Ayodeji Fagbayi	Implemented UserCredentials, UserProfile, SessionToken, registration/login persistence, PBKDF2 password hashing, profile APIs, and authentication database tests.	Focused on unique email enforcement, no plain-text password storage, one-to-one user/profile relationships, and user/administrator role checks.
Ahmad Kaseb	Implemented Service and Queue database models, service creation/update, open/close queue persistence, wait-time retrieval, schema constraints, and service tests.	Focused on required service fields, duration/priority validation, primary/foreign keys, and showing available services from database responses.
Nick Edwards Mayes	Implemented QueueEntry, Notification, and History persistence; join/leave, serve-next, remove/reorder logic; position updates; and queue/history tests.	Focused on transactionally updating queue positions and storing complete served/canceled participation history and notifications.
Ricardo Rios Hernandez	Integrated SQLite/SQLAlchemy configuration with FastAPI, updated the Assignment 2 JavaScript UI to retrieve/display database data, ran coverage, organized the repository, and prepared the final document.	Checked the full A2-A4 flow, persistence across requests, repository structure, coverage evidence, README instructions, and submission requirements.

8. Final Requirement Double-Check

Requirement	Status
Real database implemented	Complete - SQLite with SQLAlchemy; database created automatically.
Required tables/collections	Complete - UserCredentials, UserProfile, Service, Queue, QueueEntry, Notification, and History; SessionToken added for persistent authentication.

Requirement	Status
Passwords protected	Complete - salted PBKDF2-SHA256 hashes; no plain-text password storage.
Primary and foreign keys	Complete - defined in models and executable schema.sql.
Backend/database validations	Complete - required fields, length/range/type checks, unique constraints, role checks, and foreign-key enforcement.
Backend APIs store data	Complete - registration, services, queues, notifications, history, and sessions commit to SQLite.
Front-end forms submit to backend	Complete - login, registration, join queue, and service management use JavaScript fetch requests.
Data persists and is retrievable	Complete - verified by tests using separate requests and SQLAlchemy sessions.
Database data displayed in UI	Complete - available services, queue counts/status, user queue status, notifications, history, and admin queue entries.
Unit tests updated	Complete - 33 tests passed.
Coverage >= 70-80%	Complete - 92% total backend coverage.
GitHub link	Final user action required - replace the highlighted placeholder after pushing the project.
Equal team contributions	Final team action required - each member must push meaningful commits matching Section 7.
Canvas submission format	Submit one Word or PDF document only; do not upload code to Canvas.

QueueSmart Assignment 4 meets the database design, persistence, retrieval, validation, integration, and testing requirements. Only the repository URL and actual team commit history must be completed outside this document.